

Le pilotage des files de jeu

Construire une API d'analyse et un tableau de bord décisionnel à partir d'un historique de parties.

Lien du repo pour récupérer la db: https://github.com/TLouvet/R5C54_2026_SUJET

Le contexte

Riot exploite cinq jeux — League of Legends, Valorant, Teamfight Tactics, Wild Rift et Legends of Runeterra — sur six serveurs régionaux : EUW, EUNE, NA, KR, BR et LAN.

Chaque jeu propose différentes files permettant aux joueurs de trouver une partie : classée, normale, ARAM, Double Up, etc. L'équipe chargée de piloter ces files cherche à mieux comprendre leur activité.

Certaines files semblent peu utilisées. Les conserver peut disperser les joueurs et augmenter les temps d'attente ; les fermer alors qu'elles sont encore suffisamment actives peut au contraire dégrader l'expérience de jeu.

Ailleurs, les joueurs signalent des temps d'attente importants, sans que l'équipe sache immédiatement si le problème concerne un jeu entier, un serveur ou seulement certaines files. Enfin, l'activité évolue dans le temps : une baisse observée sur un serveur peut être générale ou correspondre à un déplacement de l'activité vers d'autres serveurs, jeux ou files.

L'équipe souhaite donc disposer d'un outil permettant de répondre à trois questions :

- Quelles files sont encore suffisamment utilisées, et lesquelles semblent se vider ?
- Où l'expérience de jeu se dégrade-t-elle ?
- Comment l'activité évolue-t-elle dans le temps et se répartit-elle entre serveurs, jeux et files ?

Votre mission est de construire cet outil sous la forme de deux applications séparées : une API Flask chargée d'interroger et d'analyser les données, et un tableau de bord Streamlit chargé de présenter les résultats. À vous notamment de déterminer quels indicateurs et quelles représentations permettent de répondre à ces questions.

Les données

Vous disposez du fichier `parties.db`, une base SQLite contenant environ 250 000 parties réparties sur trois ans.

Table	Colonnes	Lignes
parties	id, serveur_id, file_id, debut, attente_secondes, duree_minutes	~250 000
serveurs	id, code, nom, region	6
jeux	id, nom	5
files	id, jeu_id, nom	14

`duree_minutes` est vide lorsque la partie est encore en cours.

Une partie ne porte aucun identifiant de joueur. Les données permettent donc d'observer des parties et leur activité, mais pas de compter directement des joueurs uniques.

Les données utilisées dans ce projet sont fictives et ont été générées pour le cours.

Ce que vous devez construire

1. Une API de consultation

Votre API Flask est versionnée sous `/api/v1`. La route `GET /api/v1/parties` permet de consulter les parties.

Elle renvoie 20 parties par défaut et permet :

- de paginer les résultats avec `limit` et `offset` ;
- de filtrer par année, serveur, jeu et file ;
- de trier par date ou par temps d'attente ;
- de choisir un ordre croissant ou décroissant.

La réponse respecte le format suivant :

```
{
  "data": [...],
  "total": 83364,
  "limit": 20,
  "offset": 0
}
```

`total` représente le nombre total de parties correspondant aux filtres, et non le nombre de parties de la page courante. Les paramètres reçus doivent être validés. Une colonne de tri inconnue doit notamment produire une erreur explicite indiquant les valeurs acceptées.

2. Un accès aux données avec SQLAlchemy

L'accès final à la base est réalisé avec SQLAlchemy. Votre application distingue les responsabilités suivantes :

- le controller traite la requête et construit la réponse HTTP ;
- le service porte la logique et les validations nécessaires ;
- le repository interroge la base de données.

Le passage à SQLAlchemy ne doit pas modifier les routes ni le contrat JSON de l'API. Les sessions doivent avoir un cycle de vie maîtrisé. Les filtres utilisés par plusieurs requêtes doivent être factorisés afin qu'une liste, son total et une agrégation appliquent les mêmes règles. Les agrégations sont réalisées dans la base de données.

Les indicateurs

Vous devez choisir trois indicateurs permettant d'aider l'équipe à répondre aux questions du projet. Leur choix fait partie du travail demandé.

Pour chacun, vous devez définir :

- son nom ;
- sa formule ;
- son unité ;
- sa période ;
- sa période de référence ;
- le sens de son évolution.

L'API expose ces indicateurs avec GET /api/v1/parties/indicateurs?annee=2025.

```
{
  "periode": "2025",
  "periode_reference": "2024",
  "indicateurs": [
    {
      "cle": "kpi_1",
      "nom": "Indicateur 1",
      "unite": "unite",
      "valeur": 42.5,
      "reference": 40.0,
      "ecart": 0.0625,
      "sens": "hausse_favorable"
    }
  ]
}
```

Chaque indicateur compare l'année sélectionnée à l'année précédente. Pour 2023, première année disponible, periode_reference, reference et ecart valent null. sens indique comment interpréter une augmentation de l'indicateur : hausse_favorable ou hausse_defavorable. Sa valeur doit être cohérente avec la signification de l'indicateur choisi.

Tous les calculs sont réalisés par l'API. Le tableau de bord affiche les résultats ; il ne recalcule pas les indicateurs.

Les données d'analyse

L'API fournit les agrégations nécessaires aux visualisations :

```
GET /api/v1/parties/par-serveur
GET /api/v1/parties/par-file
GET /api/v1/parties/par-mois
```

Ces endpoints acceptent les filtres pertinents et retournent des données déjà agrégées, prêtes à être utilisées par le tableau de bord.

Un endpoint GET /api/v1/parties/referentiel fournit les serveurs, jeux, files et années disponibles.

Le tableau de bord

Vous réalisez un tableau de bord Streamlit de deux pages. Les filtres sont placés dans la barre latérale et restent cohérents lorsque l'utilisateur change de page.

Le tableau de bord permet au minimum :

- de choisir une année et de filtrer par serveur, jeu et file ;
- de consulter les parties page par page ;
- de consulter les trois indicateurs et leur évolution ;
- d'explorer les données avec trois visualisations Plotly.

Une variation positive n'est pas nécessairement une bonne nouvelle. Si un indicateur se dégrade lorsqu'il augmente, son évolution doit être présentée comme défavorable.

Les trois graphiques

Vous devez produire trois visualisations à partir des données agrégées fournies par l'API. Au-dessus de chaque graphique, indiquez en une phrase la question à laquelle il cherche à répondre.

Le choix de la représentation vous appartient. Vous devrez être capable de justifier :

- pourquoi cette donnée est pertinente ;
- pourquoi ce type de graphique permet de répondre à la question ;
- ce que l'on peut réellement conclure à partir de ce graphique.

Si un filtre ne retourne aucune partie, un message remplace le graphique. Dans une série mensuelle, un mois sans partie doit apparaître avec une valeur nulle : il ne doit pas simplement disparaître et couper la série temporelle.

Des fixtures à l'API

Le dossier fixtures/ contient des exemples de réponses JSON respectant le contrat attendu. Vous les utilisez pour commencer le développement du tableau de bord sans attendre que toute l'API soit terminée.

Dans la version finale, le tableau de bord ne lit plus ces fichiers : il appelle votre API en HTTP. Tous les appels HTTP sont centralisés dans un même module ou client. Chaque appel possède un timeout. Les erreurs HTTP et l'impossibilité de joindre l'API doivent produire un message compréhensible plutôt qu'une stack trace.

Dans la version finale, Streamlit n'importe jamais le repository, SQLAlchemy ou la base SQLite.

Progression du projet

Séance 1 — API de consultation

- Prenez connaissance du sujet
- Modéliser la DB pour avoir en tête son fonctionnement
- Mettre en place Flask et GET /api/v1/parties et la route pour obtenir les référentiels.
- Implémenter pagination, filtres, tri et validation des paramètres.
- Vérifier par exemple : `curl "localhost:5000/api/v1/parties?jeu=LoL&tri=attente&ordre=desc&limit=5"`.

Séance 2 — SQLAlchemy et indicateurs

- Porter l'accès aux données vers SQLAlchemy sans changer les routes.
- Choisir et calculer les trois indicateurs.
- Gérer la comparaison avec l'année précédente et le cas de 2023.

Séance 3 — Tableau de bord

- Construire les deux pages Streamlit à partir des fixtures.
- Mettre en place la sidebar, les filtres, la pagination et les trois indicateurs.
- À ce stade, seuls les comportements couverts par les fixtures modifient réellement les données affichées.

Séance 4 — Visualisations

- Ajouter les trois visualisations Plotly.
- Développer /par-serveur, /par-file et /par-mois.
- Traiter les jeux de données vides et les mois sans partie.

Séance 5 — Connexion à l'API

- Remplacer la lecture des fixtures par les appels HTTP.
- Centraliser les appels et ajouter un timeout.
- Tester volontairement le dashboard avec Flask arrêté.

Architecture finale attendue :

```
SQLite → SQLAlchemy → Repository → Service → Flask
      → JSON / HTTP → Client HTTP → Streamlit → pandas / Plotly
```

Hors périmètre

- écriture dans la base
- authentification ou gestion des utilisateurs
- classement ou détail individuel des joueurs
- SQL analytique avancé
- CSS Streamlit sur mesure
- appel à l'API réelle de Riot Games.

Une fonctionnalité supplémentaire ne compense pas une exigence obligatoire non réalisée.

Utilisation des LLM et agents de code

Vous pouvez utiliser les assistants conversationnels, LLM et agents de code autant que vous le souhaitez. Ils peuvent écrire une fonction, générer une requête SQLAlchemy, créer un graphique Plotly, restructurer votre projet, chercher un bug ou vous proposer une architecture.

Il n'est pas attendu que vous écriviez chaque ligne de code sans assistance. En revanche, vous êtes responsables de tout ce que vous livrez : fonctionnement, requêtes, chiffres affichés, choix d'architecture, représentations et respect du contrat de l'API.

Livrables

Le travail est individuel. Deux éléments sont attendus.

1. Le dépôt Git

Le dépôt contient l'ensemble du projet ainsi qu'un README. À partir d'un clone propre, les instructions doivent permettre d'installer les dépendances puis de lancer séparément l'API Flask et le tableau de bord Streamlit, sans avoir besoin de vous contacter.

2. La fiche technique

Deux pages maximum. Cette fiche présente vos choix et votre manière de travailler.

Les indicateurs

Présentez les trois indicateurs retenus. Pour chacun, indiquez ce qu'il mesure, comment il est calculé, son unité, sa période de comparaison et comment interpréter son évolution.

Les visualisations

Présentez les trois visualisations principales. Pour chacune, indiquez la question à laquelle elle cherche à répondre, le type de graphique choisi et pourquoi cette représentation vous semble adaptée.

Utilisation de l'IA

Expliquez comment vous avez utilisé — ou choisi de ne pas utiliser — des assistants IA pendant le projet. Vous pouvez notamment présenter les outils utilisés, les tâches pour lesquelles ils vous ont été utiles ou peu utiles, quelques exemples de prompts représentatifs, ainsi que les vérifications ou modifications que vous avez apportées aux réponses obtenues.

L'objectif n'est pas de fournir l'historique complet de vos conversations, mais de présenter concrètement la place qu'a prise l'IA dans votre manière de réaliser le projet.

Barème (indicatif)

Critère	Points	Ce qui est évalué
API Flask	5	Consultation, pagination, filtres,

		tri, validation, SQLAlchemy, indicateurs et agrégations.
Dashboard Streamlit	4	Interface, navigation, filtres, pagination, indicateurs, visualisations et gestion des cas sans données.
Fonctionnement de l'application	4	La réalisation finale fonctionne réellement de bout en bout : Flask et Streamlit communiquent en HTTP, les données ne proviennent plus des fixtures, les interactions produisent les résultats attendus et les erreurs sont gérées.
Analyse et pertinence décisionnelle	5	Les indicateurs et visualisations répondent aux questions du projet, reposent sur des calculs cohérents et permettent une interprétation pertinente des données.
Livraison et documentation	2	Projet installable et lançable depuis le README ; fiche technique claire et conforme.
Total	20	

Ce qui compte

Le projet n'est pas évalué sur le nombre de lignes que vous avez écrites vous-mêmes. Il est évalué sur votre capacité à construire une chaîne cohérente : interroger → calculer → exposer → transmettre → représenter → interpréter.

Un endpoint qui répond 200 mais retourne un mauvais chiffre n'est pas correct. Un code produit par un LLM que vous avez vérifié et correctement intégré fait pleinement partie de votre travail.